

Environnements de développement logiciel



Alexis ROYER - <https://www.linkedin.com/in/alexis-royer/>

2025-2026

Ce document constitue un cours d'ingénierie logicielle, plus particulièrement sur la mise en oeuvre des environnements de développement.

L'environnement de développement contribue à l'efficacité des activités, et par conséquent à la qualité d'un logiciel produit, d'où l'importance de maîtriser l'environnement de travail pour l'ensemble d'une équipe.

Ce document est rédigé en format Markdown suivi sous git à dessein, pour démontrer le format Markdown et comment on peut l'utiliser pour la constitution de documentations techniques, ainsi que les capacités de suivi de version et de travail collaboratif apportées par git sur un format texte.

Source : <https://github.com/alxroyer/edu-sw-dev-env/>

Sommaire :

- [1. Avant-propos](#)
 - [1.1. Types de logiciel](#)
 - [1.2. Technos et langages](#)
 - [1.3. Environnements de développement et qualité logicielle](#)
- [2. Développement](#)
 - [2.1. IDE Integrated Development Environment](#)

- 2.2. Debugging
- 2.3. Bibliothèques
- 2.4. Build
- 2.5. Industrialisation
- 3. Exécution
- 4. Assurance qualité
- 5. Delivery
 - 5.1. Enregistrements
 - 5.2. CI/CD
- 6. Déploiement

1. Avant-propos

1.1. Types de logiciel

Avant toute chose, il convient de considérer que la mise en oeuvre d'un environnement de développement est fortement dépendante du type de logiciel développé.

On distingue différents types de logiciels :

-  les applications PC (Windows, Mac, Linux),
-  les applications web (front et back),
-  les applications mobile (Android, iPhone),
-  les logiciels embarqués.

1.2. Technos et langages

Pour chacun des types de logiciels, des technologies différentes peuvent être mises en jeu (liste non exhaustive) :

-  Applications PC :
 - Langues :
 - C / C++
 - Java
 - Node.js (1) / Typescript (2)
 - Python
 - Go ? à confirmer
 - Rust (3)
 - Systèmes d'exploitation :
 - Windows
 - Mac
 - Linux
 - Frameworks :

- Qt (4)
- React Native (4)
-  Applications web :
 - Langages :
 - HTML
 - Côté *frontend* :
 - Javascript / Typescript (2)
 - Côté *backend* :
 - Node.js (1) / Typescript (2)
 - Java
 - PHP
 - Ruby, notamment avec le framework Ruby on Rails
 - Python
 - C / C++
 - Frameworks :
 - Angular (Javascript)
 - Vue.js (Javascript)
 - React Native (Javascript) (4)
 - Symfony (PHP)
-  Applications mobiles :
 - Architectures / systèmes d'exploitation :
 - Android
 - iOS
 - Langages :
 - Java, Kotlin (Android)
 - Swift (iOS)
 - Objective-C (iOS)
 - C (Code natif Android & iOS)
 - Frameworks :
 - React Native (4)
 - Kits de développement :
 - SDK Android : Software Development Kit
 - NDK Android : Native Development Kit
-  Logiciels embarqués :
 - Langages :
 - C / C++
 - Go ? à confirmer
 - Rust (3)
 - Systèmes d'exploitation :
 - Linux : couramment utilisé pour réaliser des systèmes embarqués. (5)
 - vxWorks : système d'exploitation dit *temps-réel*, propriétaire, développé par Wind River.
 - µC-OS : OS minimaliste, sans système de fichier, pile IP, ... par défaut.
 - Sans OS : simple boucle de réaction sur événement.

Notes :

- (1) Node.js : extension de Javascript, initialement restreint aux navigateurs, avec exécution par un moteur dédié.
- (2) Typescript : extension de la syntaxe Javascript pour permettre le typage.
- (3) Rust est un langage de programmation compilé, relativement nouveau, dont un des objectifs principaux est la sécurisation de la gestion de la mémoire. De fait, certains le présentent comme le successeur de C / C++.
- (4) Certains frameworks ont l'avantage d'être *cross-platform*, c'est à dire qu'ils permettent de réaliser un seul développement utilisable sur différents environnements (plateformes et/ou systèmes d'exploitation).
- (5) L'OS Linux, même dans une version embarquée, n'est pas considéré comme un OS *temps-réel*, c'est-à-dire qu'il ne garantit pas un temps d'exécution pour un traitement donné. L'occupation mémoire est plus importante que pour un OS minimaliste. En revanche, il a l'avantage de fournir un écosystème riche de services et d'outils qui peut faciliter les développements.

1.3. Environnements de développement et qualité logicielle

La norme ISO 9126, remplacée aujourd'hui par l'ISO 25010, identifie différentes dimensions pour la caractérisation de la qualité d'un logiciel :

1. Capacité fonctionnelle
2. Facilité d'utilisation
3. Fiabilité
4. Performance
5. Maintenabilité
6. Portabilité

Une partie de ces dimensions reste essentiellement portée par le logiciel développé.

Toutefois, l'environnement de développement peut contribuer à certaines de ces dimensions, notamment :

- Fiabilité :
 - Mettre en oeuvre d'outils d'analyse de code statique, permettant d'éviter des bugs avant même d'exécuter le code.
 - Exécuter régulièrement des tests de non-régression, pour détecter les problèmes au plus tôt.
- Maintenabilité :
 - Assurer un maximum de confort pour les développeurs, de sorte à faciliter le codage, le debugging... de par le choix des outils pour leurs fonctionnalités, et sans oublier leurs performances ! Faire en sorte de limiter les freins aux différentes actions nécessaires à la réalisation d'un logiciel de qualité.
 - Utiliser un outil de contrôle de version (git).
 - Automatiser les vérifications de qualité (linters, indentation) pour s'assurer d'un certain niveau de lisibilité du code, donc un code plus facile à maintenir, à suivre en historique.
- Portabilité :
 - Choix de technos cross-platform (à la frontière avec l'environnement de développement).
 - Exécution des tests de non-régression sur les différentes cibles.

2. Développement

Avant de délivrer et exécuter le logiciel, commençons par mettre en oeuvre les éléments permettant de le construire.

2.1. IDE (*Integrated Development Environment*)

Un des premiers outils qu'on est amené à aborder pour faire du développement logiciel est l'IDE.

L'usage d'un IDE présente de nombreux intérêts :

- Edition de code : Basique, un simple notepad pourrait faire le job.
- Coloration syntaxique : Ca change déjà un peu la vie, dans la mesure où ça fluidifie la lecture du code.
- Complétion : On commence à gagner du temps, en évitant des fautes de frappe donc des erreurs au passage.
- Refactoring : On gagne encore plus du temps. On n'hésite plus à renommer une variable, une fonction, une classe, pour lui redonner un nom plus adapté, allant dans le sens de la maintenabilité.
- Debugging : Dès lors que c'est possible, permet souvent d'obtenir plein d'informations pour la résolution de bugs.
- IA : Les IDE intègrent aujourd'hui des fonctions d'IA qui peuvent augmenter la productivité.

⚠ Attention : IA et confidentialité

A ce jour, l'IA fonctionne principalement par alimentation d'informations sur des moteurs s'exécutant sur Internet.

L'utilisation de l'IA dans les IDE peut donc être problématique si l'entreprise ne souhaite pas divulguer le code.

Bien se renseigner sur les usages possibles ou non.

Possible que l'entreprise dispose d'un contrat de confidentialité avec une IA donnée, voire héberge une solution d'IA *on premise*, auquel cas on pourra configurer les IDE pour utiliser cette IA validée par l'entreprise.

On liste ci-après des IDE connus (liste non exhaustive) :

-  Applications PC :
 - Visual C++
 - Eclipse : Java, mais aussi C / C++ (version CDT)
 - VS Code : Javascript, mais aussi C++ (extension)
 - JetBrains PyCharm : Python
-  Applications web :
 - VS Code : HTML, Javascript, Typescript, PHP
 - PHPStorm
 - JetBrains IntelliJ : Java
 - JetBrains IDEA Ultimate : tous langages

- Kustomize : Docker, Kubernetes
-  Applications mobile :
 - Android Studio
 - JetBrains IntelliJ : Java, Kotlin
 - XCode : iOS
-  Logiciel embarqué :
 - VS Code : C / C++ (extension), Rust (extension)
 - Eclipse : C / C++ (version CDT)

 TP : [Configuration d'un IDE VS Code pour C/C++](#)

2.2. Debugging

① Note

On parle de debugging dès à présent, car c'est une des fonctions majeures apportées par un IDE facilitant l'investigation et la résolution de bugs logiciels.

Cela entend qu'on sait exécuter notre logiciel au préalable (cf. § [Exécution](#)). La notion reste toutefois abordable en considérant le cas simple des applications PC dans un premier temps.

Les debuggers (tels [gdb](#)) peuvent généralement s'utiliser en ligne de commande. Toutefois, cela reste extrêmement compliqué, et l'utilisation d'une interface graphique est quasiment indispensable pour déboguer efficacement.

Les IDE intègrent très bien les debuggers, et permettent ainsi de basculer très rapidement entre les modes édition de code et debugging / intégration.

Parmi les fonctions utiles d'un debugger, on note :

- la possibilité de positionner des points d'arrêt (*breakpoints*)
- l'exécution pas à pas, i.e. ligne à ligne,
- l'observation des valeurs de variables,
- rentrer dans l'exécution des fonctions (et en ressortir)
- l'affichage de la pile d'appels (*callstack*)
- la possibilité de définir un point d'arrêt sur modification de valeur : très utile pour investiguer des problèmes de débordement mémoire
- la capacité à modifier des valeurs de données : utile pour tester des injections de fautes
- la capacité pour certains langages interprétés (Python, JS) à pouvoir exécuter des routines à chaud
- ...



 TP : [Debugging C/C++ avec VS Code](#)

Dans le cas du debugging d'un logiciel embarqué, la difficulté est de savoir synchroniser un IDE/debugger qui tourne sur une machine de développement, et le programme à déboguer qui tourne sur une cible autre. Pour ce faire, on peut utiliser des moyens spécifiques :

- [gdbserver](#) :

- Possible pour des cibles tournant sous Linux uniquement.
- Le programme à débbugger est lancé sur la cible avec **gdbserver**, et le debugger sur la machine de développement se connecte au **gdbserver** en tant que client.
- J'ai déjà constaté personnellement que la connexion pouvait être instable. Pas de garantie notamment que les points d'arrêts soient bien pris en compte, notamment dans des cas d'applicatifs multi-threadés.
- Utilisation d'une sonde :
 - Matériel de sonde spécifique au micro-processeur de la cible.
 - Branchement sur un bus série dédié, directement sur le micro-processeur sur la cible. Nécessite que les signaux aient été routés sur la carte.



TP : Debugging C/C++ à distance avec gdbserver (TODO)

On note finalement que les investigations par debugging ont leurs limites :

- Pas de debugging possible *post-mortem* tant qu'on n'a pas caractérisé les conditions de reproduction d'un problème.
- Il arrive fréquemment que le fait de poser des points d'arrêts provoque des effets de bords sur des timeouts qui arrivent à échéance, et donc des effets indésirables empêchant un debugging confortable.
- Dans le cas des logiciels embarqués, comme le logiciel s'exécute sur une cible autre, le debugging est par nature moins facile à mettre en oeuvre.
- Cas en Python où le débbugueur exécute automatiquement des routines **__repr__()** pour obtenir des représentation des objets à afficher dans l'interface, ces exécutions surnuméraires pouvant provoquer des effets de bords indésirables selon les cas.

En raison de ces limitations, la bonne vieille technique du debugging "à la trace" ne doit pas être négligée.

Plus encore, il convient de mettre en place un système de trace efficace dès le début du projet, avec quelques fonctions utiles :

- Niveaux de criticité : debug, info, warning, erreur classiquement.
- Colorisation : savoir faire apparaître les erreurs en rouge a minima.
- Indentation : utile pour analyser les logs de traitements récurifs.
- Filtrages : par fonction, module.

2.3. Bibliothèques

2.4. Build

2.5. Industrialisation

La constitution d'images Docker embarquant des versions d'outils bien identifiées contribue à la maîtrise de l'environnement de développement.

Grâce à ces images, on peut répéter facilement une configuration de développement, pour chaque membre de l'équipe, ainsi que pour la chaîne de CI/CD, et avec l'assurance d'utiliser les mêmes versions des outils.

3. Exécution

Dès les premières versions de notre logiciel disponibles, il s'avère intéressant de savoir exécuter notre production aux différentes étapes de sa construction :

- en cours de développement, pour des tests unitaires, ou tests d'intégration libres,
- sur des versions intermédiaires, pour des tests de non-régression quotidiens par exemple,
- sur des versions identifiées, pour des campagnes de test avant livraison.

4. Assurance qualité

5. Delivery

5.1. Enregistrements

5.2. CI/CD

6. Déploiement
